

“Intelligent Water Distribution Monitoring and Leakage Detection Platform”

CO1 Assessment Tool 2

Submitted in the partial fulfilment for the Course of

CSA1016 - Software Engineering

*to the award of the
degree of*

**BTECH
(Information Technology)**

Submitted by

192321050- Lenin K

**Under the Supervision
of Dr. KAnita Davamani**



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING

**Saveetha Institute of Medical and Technical
Sciences Chennai-602105**



SIMATS ENGINEERING

Saveetha Institute of Medical and Technical
Sciences Chennai-602105



DECLARATION

I, **Lenin K 192321050**, of the **Department of Information Technology**, Saveetha Institute of Medical and Technical Sciences, Saveetha University, Chennai, hereby declare that the Capstone Project Work entitled “**Collaborative Story/ Screenplay Writing Platform with Version History**” is the result of our own Bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with principles of engineering ethics.

Place: Chennai

Date:

Signature of the Students

INDEX TABLE

S.No.	Architecture Design Report Sections
1	System Requirements Analysis (Objectives, Functional & Non-Functional, Quality Attributes)
2	Selection of Software Architecture (Style & Comprehensive Technical Justification)
3	Software Architecture Diagram (System Components, Communication & APIs)
4	Component Description and Interactions (Responsibilities & System Workflow)
5	Evaluation of Key Quality Attributes (Scalability, Security, Performance, etc.)
6	Justification of Architectural Decisions & Trade-Offs (Rationale & Enhancements)
	Appendix: Evaluation Rubrics Matrix (Self-Assessment)

1. Analyze System Requirements

The target system is an Intelligent Water Distribution Monitoring and Leakage Detection Platform. It serves as a city-scale monitoring backbone that continuously observes a municipal water pipeline network through distributed IoT sensors, automatically identifies leaks and abnormal consumption using anomaly-detection models, and coordinates field response through GIS-linked alerts and work orders.

1.1. System Objectives

- **Real-Time Telemetry Monitoring:** Continuously ingest flow, pressure, and acoustic vibration readings from sensors installed along the pipeline network with sub-minute end-to-end latency.
- **Automated Leak & Anomaly Detection:** Apply statistical thresholds and machine-learning models to sensor streams to detect leaks, bursts, and abnormal consumption without manual inspection.
- **GIS-Based Network Visualization:** Represent the pipeline network, valves, junctions, and sensor placements on an interactive geospatial map, overlaying live sensor status and detected anomalies.
- **Alerting & Work-Order Automation:** Automatically classify leak severity, generate prioritized alerts, and dispatch geo-located work orders to field technicians.
- **Non-Revenue Water (NRW) Analytics:** Track historical water-loss trends, sensor uptime, and maintenance KPIs to support long-term infrastructure planning.
- **Interoperability with Municipal Systems:** Integrate with existing SCADA systems, billing platforms, and GIS asset registries used by the water utility.

1.2. Functional Requirements (Influencing Architecture)

Functional requirements dictate the structural abstractions of the system. The table below lists the key functional areas that directly shape the architectural layout:

Functional Module	Requirement Details	Architectural Influence
IoT Telemetry Ingestion	Thousands of flow/pressure/acoustic sensors must publish readings every 2-10 seconds with guaranteed delivery, even over unreliable field connectivity.	Requires an MQTT broker cluster with QoS guarantees, edge gateways for local buffering, and a high-throughput stream-ingestion layer (Kafka).
Leak Detection Engine	Sensor streams must be continuously analyzed to flag pressure drops, unexplained flow, and acoustic leak signatures within seconds of occurrence.	Requires a dedicated ML inference microservice consuming from Kafka, backed by a time-series database for baseline comparison.
GIS & Pipeline Mapping	Users must view pipeline segments, valves, sensor health, and live leak markers on an interactive map, and query which segment is affected.	Requires a geospatial database (PostGIS) and a dedicated GIS microservice with spatial indexing.
Alerting & Work-Order Mgmt	Detected leaks must generate severity-classified alerts, notify technicians, and track resolution status end-to-end.	Requires an event-driven alerting microservice, push/SMS/email integration, and a work-order datastore.
Historical Analytics & Reporting	Utility managers need dashboards on non-revenue water %, sensor uptime, and repair turnaround time over weeks/months.	Requires a time-series database, scheduled batch-aggregation jobs, and a reporting microservice.

Functional Module	Requirement Details	Architectural Influence
Device Provisioning & OTA	Field technicians must register new sensors, monitor battery/health status, and push firmware updates remotely.	Requires a device-registry microservice and an MQTT device-shadow pattern.

1.3. Non-Functional Requirements (NFRs)

NFR Category	Target Metric	Architectural Design Decision
Latency	Leak alert raised within 15 seconds of anomaly onset.	Stream processing (Kafka Streams) co-located with ML inference; no batch delay.
Throughput	Support 200,000+ concurrent sensor connections city-wide, each transmitting every 5 seconds.	Horizontally scalable MQTT broker cluster (EMQX) with topic sharding.
Availability	99.9% uptime for the ingestion and alerting pipeline.	Multi-AZ Kubernetes deployment, Kafka replication factor 3, database failover replicas.
Data Durability	Zero telemetry loss during network partitions.	Edge gateways buffer readings locally and resend on reconnect; MQTT QoS 1/2 delivery.
Security	Encrypted device-to-cloud communication and role-based access for utility staff.	TLS 1.3 on MQTT/HTTPS, mutual-TLS device certificates, OAuth2/JWT for staff, RBAC scopes.
Scalability	Elastic support for new pipeline zones/districts without redesign.	Stateless microservices behind an API gateway; Kafka partition scaling; sharded time-series DB.

1.4. Key Quality Attributes

To provide a solid software architecture, we identify and optimize six core quality attributes:

- **Scalability:** The platform must scale from a single district pilot to hundreds of thousands of concurrent sensor connections city-wide. The architecture isolates memory-heavy MQTT ingestion from persistent databases using stateless gateways and partitioned event streams.
- **Maintainability:** Each sub-domain (Device Management, Leak Detection, GIS, Alerting, Analytics) is developed as a separate microservice, allowing teams to refactor or redeploy one domain without risking global regression.
- **Reliability:** Data integrity is critical for public infrastructure decisions. Edge buffering, MQTT QoS guarantees, and Kafka replication ensure telemetry and leak events are never silently lost.
- **Availability:** Redundant containerized nodes, automated service discovery, and database failover strategies guarantee continuous monitoring even if a database primary node fails.
- **Performance:** Field technicians and operators expect near-immediate leak alerts. Anomaly detection runs as a streaming pipeline rather than a scheduled batch job, and heavy reporting tasks run asynchronously in the background.
- **Security:** Requires deep transport-layer protection (TLS 1.3), mutual-TLS device authentication, role-based authorization scopes for utility staff, and encrypted storage of pipeline asset data.

2. Select a Suitable Software Architecture

We select a Hybrid Software Architecture to satisfy the divergent constraints of massive-scale IoT telemetry ingestion, real-time anomaly detection, geospatial querying, and resilient field-response coordination.

The hybrid system combines three architectural patterns:

- **1. Edge Computing:** Lightweight pre-processing and buffering occur at field gateways before telemetry reaches the cloud, reducing bandwidth usage and tolerating intermittent rural/underground connectivity.
- **2. Event-Driven Architecture (EDA):** An MQTT + Kafka backbone decouples telemetry ingestion from downstream processing. Leak-detection results, alerts, and work-order events propagate asynchronously through the event bus without blocking device connections.
- **3. Microservices Architecture:** Domain logic — Device Management, Leak Detection & ML Inference, GIS & Mapping, Alerting & Notification, and Analytics & Reporting — is isolated into independently deployable, independently scalable services, fronted by a single API gateway for client traffic.

2.1. Technical Justification for the Selected Hybrid Architecture

Architectural Need	Monolith / Basic MVC Limitations	Hybrid (Edge + EDA + Microservices) Advantage
High-frequency sensor ingestion at city scale	A single monolithic server writing every sensor reading synchronously would exhaust database connections and create write bottlenecks under hundreds of thousands of devices.	MQTT brokers absorb device fan-in; Kafka buffers and replays telemetry, decoupling ingestion rate from downstream processing capacity.
Real-time leak detection without blocking ingestion	Running ML inference inline within the request/response cycle would stall telemetry ingestion during model execution.	Leak Detection runs as an independent Kafka consumer group that scales horizontally and never blocks the ingestion path.
Heavy GIS / spatial queries	Spatial joins across a large pipeline network on a general-purpose relational schema are slow and lock-prone in a monolith.	A dedicated PostGIS-backed GIS microservice with spatial indexes isolates heavy geo-queries from transactional workloads.
Resilience against partial failure	A crash in the reporting module of a monolith can bring down live leak alerting for the whole city.	Each microservice fails independently; a crash in the Analytics Service does not affect Leak Detection or Alerting.
Field connectivity gaps	A pure cloud-only design loses data when rural or underground sensor nodes lose connectivity.	Edge gateways buffer readings locally and forward once connectivity resumes, guaranteeing durability.

3. Draw the Software Architecture Diagram

The diagram below represents the software architecture. It illustrates the field sensor layer, ingestion pipeline, API routing, microservice decomposition, message streaming, and persistent data stores that together manage the water distribution monitoring workflow.

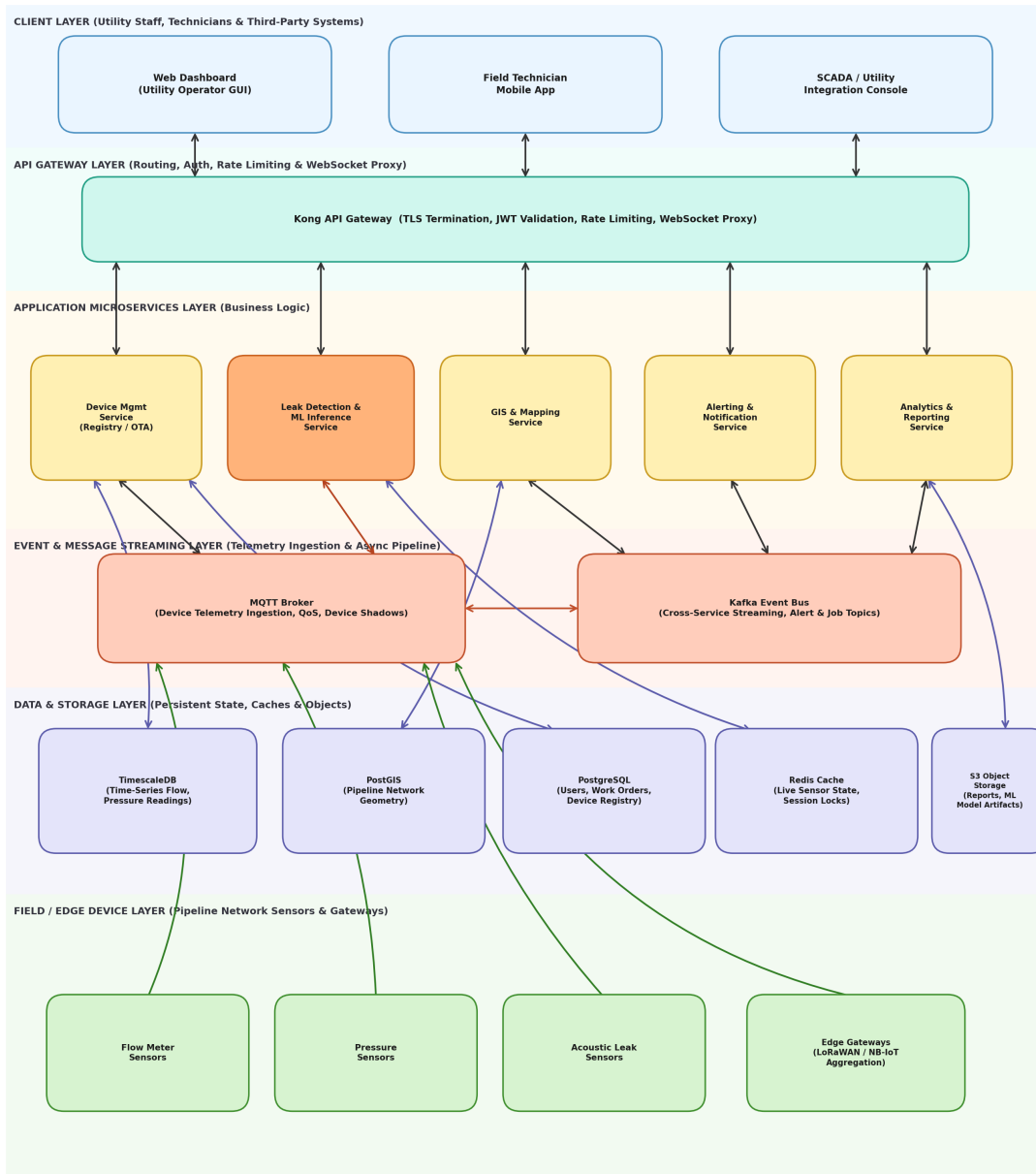


Figure 3.1: Complete Software Architecture Blueprint for the Water Distribution Monitoring Platform

The architecture decomposes the platform into distinct, independent tiers. Below, we describe the integration paths:

- Field sensors and edge gateways publish flow, pressure, and acoustic telemetry over MQTT (QoS 1/2, TLS-secured) to the ingestion layer, tolerating intermittent connectivity through local buffering.

- The MQTT Broker fans telemetry into the Kafka Event Bus, which decouples ingestion throughput from downstream microservice processing and enables replay for auditing or model retraining.
- Clients (Web Dashboard, Field Technician Mobile App, SCADA Integration Console) establish REST and WebSocket connections through the Kong API Gateway, which terminates TLS, validates JWT tokens, and enforces rate limits.
- The Leak Detection & ML Inference Service consumes telemetry from Kafka, compares live readings against per-segment time-series baselines in TimescaleDB, and publishes leak.detected events back onto Kafka.
- The Alerting & Notification Service consumes leak events, resolves the affected pipeline segment via the GIS & Mapping Service (PostGIS), classifies severity, creates a work order in PostgreSQL, and pushes a real-time alert to the dashboard over WebSocket.
- Persistent storage is split by workload: TimescaleDB (time-series sensor readings), PostGIS (pipeline network geometry), PostgreSQL (users, work orders, device registry), Redis (live sensor state and locks), and S3-compatible object storage (reports and ML model artifacts).

4. Explain Components and Their Interactions

A comprehensive description of individual system components and their communications governs the architectural soundness of this platform.

4.1. Component Roles and Responsibilities

Component Name	Core Responsibilities	Technologies
Sensor Nodes & Edge Gateway	Captures flow, pressure, and acoustic data; performs local filtering/compression; buffers and forwards readings via MQTT during outages.	Flow/pressure transducers, piezoelectric acoustic sensors, ESP32/Raspberry Pi gateway, LoRaWAN / NB-IoT
MQTT Broker	Receives telemetry from all field devices, enforces QoS delivery guarantees, and manages device shadows / last-known-state.	EMQX / Mosquitto cluster, TLS 1.3, mutual-TLS device auth
Kafka Event Bus	Buffers telemetry streams and cross-service events (leak.detected, workorder.created), enabling replay and decoupled scaling.	Apache Kafka, Schema Registry
Kong API Gateway	Terminates TLS for client traffic, validates JWT tokens, enforces rate limits, and proxies WebSocket connections for live map updates.	Kong, Nginx
Device Management Service	Registers and provisions sensors, tracks battery/connectivity health, and manages OTA firmware rollout.	Node.js, PostgreSQL
Leak Detection & ML Inference Service	Consumes telemetry streams, compares readings against time-series baselines, runs anomaly-detection/ML models, and publishes leak events with severity and confidence scores.	Python, Kafka Streams / Flink, TensorFlow, TimescaleDB
GIS & Mapping Service	Maintains pipeline network geometry, resolves which segment or zone a leak event belongs to, and serves map-query APIs.	Python / Node.js, PostGIS, GeoServer
Alerting & Notification Service	Classifies severity, creates work orders, pushes real-time alerts to dashboards and mobile apps, and sends SMS / email / push notifications.	Node.js, WebSocket server, Twilio / Firebase Cloud Messaging
Analytics & Reporting Service	Aggregates historical telemetry into non-revenue-water metrics, sensor uptime, and maintenance KPIs; generates scheduled reports.	Python, Apache Airflow, TimescaleDB

4.2. Component Interactions and Data Flow

Communication protocols are optimized for each channel's requirements:

- **MQTT (Device → Cloud):** Used exclusively for field-sensor telemetry. A single persistent, QoS-guaranteed connection carries flow, pressure, and acoustic readings from each edge gateway to the broker.
- **Kafka (Service ↔ Service):** Handles asynchronous cross-service events. When the Leak Detection Service flags an anomaly, it publishes a leak.detected event; the Alerting Service and Analytics Service both consume it independently without direct coupling.

- **HTTP/REST with JSON:** Used for CRUD operations such as managing device registrations, updating pipeline metadata, and retrieving historical reports — standard client-server messaging for non-real-time interactions.
- **WebSocket (WSS):** Used to push live leak alerts and map-pin updates to the Web Dashboard and Mobile App the moment an anomaly is confirmed, without requiring the client to poll.
- **gRPC / Internal REST:** Used for internal microservice communication (e.g., the Alerting Service querying the GIS Service for segment resolution), using short-lived, low-latency internal calls.

4.3. System Workflows & Sequence Flow

The sequence below illustrates the real-time leak-detection workflow, tracking a sensor reading from the field through anomaly detection to alert delivery and work-order dispatch:

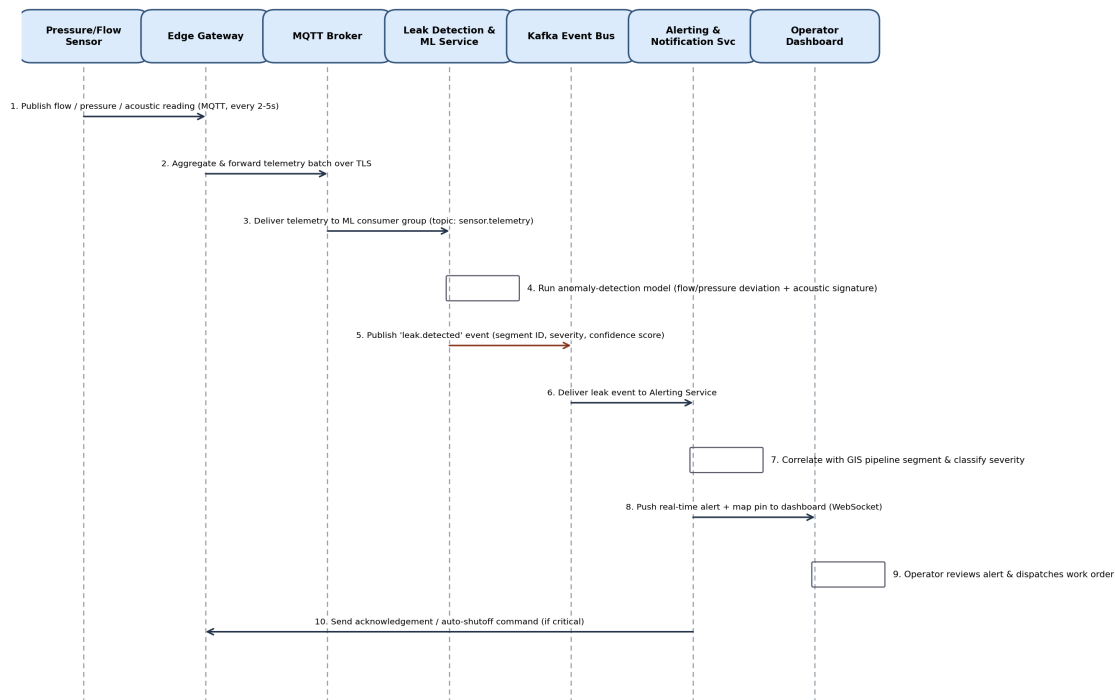


Figure 4.1: Real-Time Leak Detection — Sensor Reading to Work-Order Workflow

Figure 4.1: Real-Time Leak Detection — Sensor Reading to Work-Order Workflow

5. Discuss Quality Attributes

Evaluating how our architecture addresses critical quality metrics helps verify system viability:

Quality Attribute	Architectural Strategy	Concrete Implementation Details
Scalability	Horizontal scaling of MQTT/ingestion nodes, sharded time-series storage, and isolation of heavy services.	MQTT broker nodes and Kafka partitions scale independently behind a load balancer. TimescaleDB hypertables are chunked by time and sharded by pipeline zone to prevent indexing bottlenecks as new districts are onboarded.
Security	Securing device and client connections, sandboxing processes, and maintaining access logs.	All device connections use mutual-TLS certificates; all client connections run TLS 1.3. JWT tokens are validated at the Gateway, and the ML Inference container runs in an isolated sandbox to prevent injection through malformed sensor payloads.
Performance	Streaming inference, edge pre-filtering, and background processing for heavy tasks.	Anomaly detection runs continuously as a Kafka Streams job rather than a scheduled batch, so alerts are raised within seconds. Report generation and model retraining run as background Airflow jobs, keeping the live pipeline responsive.
Reliability	Guaranteed-delivery messaging, database replication, and edge-side buffering.	MQTT QoS 1/2 with edge-side local buffering ensures no telemetry is lost during connectivity gaps. TimescaleDB and PostgreSQL run with replica sets and automated failover.
Maintainability	Enforcing service isolation, clean API contracts, and automated testing pipelines.	Services are separated by domain boundary (Device Mgmt, Leak Detection, GIS, Alerting, Analytics), communicate via OpenAPI-documented REST/Kafka schemas, and are validated by CI/CD pipelines before deployment.
Availability	Multi-region failover, health checks, and auto-healing container orchestration.	Kubernetes health probes automatically restart unresponsive service pods. Database replica sets promote a secondary node automatically if the primary fails, keeping monitoring and alerting online.

6. Justify Architectural Decisions

Every architectural structure represents a balance of constraints. Below, we justify our choices, discuss the trade-offs considered, and outline our plans for future scalability.

6.1. Rationale Behind Design Choices

Our core architectural choice is the separation between high-frequency field telemetry ingestion and the analytical/relational systems that consume it. Using MQTT for device connectivity and Kafka as an internal event backbone ensures that sensor ingestion never blocks on database writes or ML inference. By offloading anomaly detection to an independently scaled streaming consumer and heavy reporting tasks to background jobs, we maintain a responsive alerting pipeline even as the sensor fleet grows into the hundreds of thousands.

6.2. Trade-off Analysis Matrix

Decision	Selected Path	Advantages Gained	Drawbacks / Mitigations
Telemetry Ingestion Protocol	MQTT over raw HTTP polling	Lightweight pub/sub suited for constrained IoT devices; persistent connections reduce overhead; built-in QoS for lossy field networks.	Requires broker-cluster operational expertise. Mitigated by using a managed broker service (e.g., AWS IoT Core / managed EMQX).
Anomaly Detection Approach	Hybrid statistical threshold + ML model per pipeline segment	Statistical thresholds catch obvious bursts instantly, while ML models capture subtle slow leaks; per-segment baselines reduce false positives.	Requires periodic model retraining per segment. Mitigated via a scheduled retraining pipeline with drift monitoring.
Database Selection	Polyglot storage (TimescaleDB + PostGIS + PostgreSQL)	Each store is optimized for its workload — time-series compression, spatial indexing, and relational integrity respectively.	Increases operational complexity. Mitigated by containerized deployment and unified observability tooling.
Edge Pre-processing	Local filtering/buffering at the gateway vs. raw continuous streaming	Reduces bandwidth cost, tolerates intermittent connectivity, and offloads noise filtering from the cloud.	Adds firmware complexity on constrained devices. Mitigated via an OTA update pipeline and a standardized edge SDK.

6.3. Future System Integration & Long-Term Maintainability

Our chosen architecture supports future enhancements through its modular API design. The API gateway separates external client interfaces from internal services, allowing new capabilities — such as water-quality sensing, predictive maintenance scoring, digital-twin simulation of the pipeline network, or direct billing-system integration — to be added as new microservices without modifying the core ingestion or leak-detection pipeline. All service contracts are defined using OpenAPI and Kafka schema definitions, allowing teams to evolve components independently, in different languages if needed, as the platform expands to additional utility districts.

Appendix: Architecture Design Assignment Rubric Matrix

This section reproduces the assessment rubric for evaluation reference:

Evaluation Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Needs Improvement (1)
System Requirement Analysis (30%)	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture (25%)	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram & Component Design (20%)	Produces a clear, complete, and well-structured diagram with all major components, interfaces, and interactions accurately represented.	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and lacks sufficient detail.	Diagram is incomplete, unclear, or technically incorrect.
Component Interaction & Quality Attribute Analysis (15%)	Clearly explains component responsibilities, interactions, and thoroughly discusses all six quality attributes.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.
Architectural Justification & Technical Presentation (10%)	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-organized report.	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope.	Justification is weak or absent; report lacks organization and technical clarity.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25